CST1500 – Laboratory Worksheet - Week 8

These weekly laboratory worksheets provide information and exercises to complete during class or at your own leisure. The practical exercises will assist you with your coursework as well as provide you with valuable knowledge and skills. Please do your best to keep up with the weekly laboratory worksheets in order to maximise your potential for good marks and satisfactory progress in the module.

Exercises are denoted in red writing. For example:

EX 1: Identification of PC parts (identify as many as you can).

Contents

Lectures videos to watch this week 1

Test 2 1

Selection (if, then, else) in Python Programming 1

Lectures videos to watch this week

Lecture 16 Memory Management
Lecture 17 Virtual Memory

Test 2

You may take the FORMATIVE Test 2 to test yourself on what you have covered in the lectures. It covers material from Lecture 07 – Lecture 13 inclusive. This would be useful for your own development as well as serving as an indicator of your progress to the teaching team. Your lab tutor will provide you with the specified time for the test in the lab.

Selection (if, then, else) in Python Programming

Sometimes you will change what you do depending on the conditions.

For example: IF you wake up in the morning and it is raining THEN you will take a coat to school, OTHERWISE you won't.

IF the day is a Saturday AND the alarm clock goes off THEN you might turn it off and stay in bed, OTHERWISE you might get up.
Life is full of decisions that you will make depending on certain conditions, computers are no different.

if-else

For a computer to make decisions based on a condition, you must use an IF statement, it has the following structure:

```
if condition:
     true
     several instructions that are executed
     if the calculation evaluates to True
else:
     false
     several instructions that are executed
     if the condition evaluates to False
```

EX 1: Understanding the conditional programs in Python

Consider the following IF statement:

```
age = int(input("Enter your age "))
if age >= 18:
     print("You are an adult")
else:
     print("You are still a child")
```

The IF statement explained:

• after the if is the condition age >= 18 followed by a colon, this is checking to see if the age variable is more than or equal to 18.

• after that line is the code that will only be run (executed) if that condition is True. If it is true it will print You are an adult on screen.

• the word else is then followed by a colon. The instructions underneath this are what will be executed only if that condition is False. If it is false, it will print You are still a child on screen.

Show your output to your lab tutor, SLA or classmate.

if-elif-else

An IF statement with an else will only allow you to check a single condition, however if you have more than one condition to check you can use if..elif..else

Consider the following IF statement:

```
colour = input("Enter the colour of the traffic light ")
if colour == "Red":
     print("STOP")
```

```
elif colour == "Amber":
     print("GET READY TO STOP")
else:
     print("GO")
```

The IF statement explained:

• the program first checks to see if the colour entered is Red and if it is will print STOP.
• if the colour entered isn't red it will go onto the next condition where the elif is and check if the colour is Amber. If it is then it will print GET READY TO STOP
• if neither condition is met, it will go to the else part of the code and print GO.

NOTE: It doesn't need to use elif to see if the colour is Green as if it isn't Red or Amber it must be Green, therefore you can just use else if there are no other options to consider.

Show your output to your lab tutor, SLA or classmate.

Example: Grade Calculator Program

The code for the program below will ask the user to enter a score on a test and then display what grade they got.

```
score = int(input("Enter a score between 0 and 100: "))
if score >=70:
     print("That test score is a grade A")
elif score>=60:
     print("That test score is a grade B")
elif score>=50:
     print("That test score is a grade C")
elif score>=40:
     print("That test score is a grade D")
else:
     print("That test score is a grade U")
```

NOTE: When elif is used it will only check the next condition if the previous condition is false.

Show and discuss your output with your tutor.

EX 2: Practice the if statement in Python

Task 1: Write a program in Python that will ask the user the capital city of England and display a suitable message depending on whether they enter London.
Expected Output

```
>>>
What is the capital city of England? London
Correct the capital city of England is London
>>> ================================ RESTART ====
>>>
What is the capital city of England? Paris
Wrong, try again
>>>
```

Task Two: Write a program in Python that can easily do a type check to discover whether a value is a number or not a number. You may use the function variable.isdigit() which will return a value of either True or False after checking to see if a variable is a number. You can then use an If to determine what to do.

Expected Output

```
>>>
Enter a number 1234
You entered a number
```

Number entered

```
>>>
Enter a number Hello
You didn't enter a number
```

Text entered

EX 3: Creating advanced programs in Python

You can validate data by making sure it is within a certain range. Write a program that will allow the user to enter a test score which must be between 0 and 100.

Expected Output

```
>>>
Please enter a number between 1 and 100: 123
You must enter a number between 1 and 100
```

Entered out of range

```
>>>
Please enter a number between 1 and 100: 78
You entered a valid number
```

Entered within range

FOR Loops in Python

A for loop is known as a count-controlled loop, you should use it when you want to repeat a block of code for a set number of times.

```
print("Hello everyone")
print("Hello everyone")
print("Hello everyone")
print("Hello everyone")
print("Hello everyone")
```

Without a loop

```
for x in range(0,5):
    print("Hello everyone")
```

Using a loop

How the for loop works

• A for loop will repeat for a set number of times and it will repeat between two ranges. range(min_value,max_value) will repeat between the min and the max value but not include the max value.
• range(1,5) will repeat 4 times, 1,2,3,4 but not 5 as it doesn't repeat for the maximum value.
• x is just a variable that is a counter and stores the number in the loop, this is useful if you need to use it in calculations.

EX 4: Understanding a basic for loop program in Python

Example: Name Repeater Program

This program asks the user to enter their name and their age. It will then print their name using their age as the max value. Show and discuss your output with your lab tutor, SLA or classmate.

```
name = input("Enter your name ")
age = input("Enter your age ")
#the loop uses the age entered as the maximum value in the loop
for x in range(0,int(age)):
      print("Your name is " + name)
```

Example: Times table Program

This program will ask the user what times table they want to learn, it will then calculate the times tables for that number. It makes use of the counter (x) in the loop to calculate the answer.

```
timesTable = input("Enter the times table you wish to learn (1-12)")
# loop that will repeat 13 times (0-12)
for x in range(0,13):
      # the answer uses x which increases each time to work out the
      answer
      answer = x * int(timesTable)
      # x is used in the print command to display what it is multiplying
      the number entered by
      print(timesTable + " x " + str(x) + " = " + str(answer))
```

What is the expected output? Discuss your output with your lab tutor, SLA or classmate.

EX 5: Creating advanced code in Python which combines the use of if with a for loop.

Write a calculator program which asks the user to enter a five-digit number. It then uses sub-strings to add digits 1, 3 and 5 together and subtract digits 2 and 4. This program combines using an if statement with a for loop. The program should simply do the following: (digit 1 + digit 3 + digit 5) – (digit 2 + digit 4) = output.

Expected Output

```
Enter a five digit number: 53629
The final total is: 15
>>>
```

The answer to this example is 15. This is because it will add the first, third and fifth digit to the total (5 + 6 + 9) which gives 20. It then subtracts digits 2 and 4 (3 + 2) from the total, therefore 20 - 5 = 15.

Iteration - Condition Controlled

Iteration is the process of repeating a process over and over. Often in programming you need to repeat a block of code several times.

EX 6: Understanding the while loop program

A while loop is known as a condition-controlled loop, you should use it when you do not know how many times the code needs to repeat as you can say repeat while a condition is True.

```
userentry="y"
while userentry!="n":
     userentry=input("Play again? y/n ")
print("Game over")
```

Expected Output

```
Play again? y/n y
Play again? y/n y
Play again? y/n y
Play again? y/n n
Game over
>>>
```

How the while loop works

• there is a condition after the word while, it works like an if condition. while the variable userentry is not equal to n the code inside the loop (that is indented) will repeat.
• when n is entered by the user, the loop will end and it will continue with the code after the loop (not indented). In this case it will print “Game Over”.

Example1: Guess the Number

This program asks the user to guess the number, it will keep asking them to guess the number until they guess it correctly. Once they have guessed it correctly it will tell them how many attempts it took. Discuss your output with your lab tutor, SLA or classmate.

```
answer = 15
attempts= 0
userentry=""

#a loop that repeats while the users guess is not the same as
the answer
while answer != userentry:
     userentry = int(input("Enter a number between 1 and 20 "))
     #each time through the loop 1 is added to the number of
     attempts
     attempts=attempts+1
#after the loop it will say how many attempts it took
print("Well done you correctly guessed the number it took you "
+ str(attempts) + " attempts")
```

Example 2: Adding User Numbers Program

This program asks the user to enter a number. It then asks them whether they want to enter another. If they do, it will ask them for another number and add it to the previous number. It will keep doing this until they say they do not want to enter any more numbers. Finally, it will output the total.

```
total = 0
another = "Y"
#the loop will repeat while the user types Y when asked if they
want to enter another number
while another == "Y":
     #asks the user to enter a number
     number = int(input("Enter a number to add to the total: "))
     #adds the number entered to the total
     total = total + number
     #asks the user if they want to enter another number
     another = input("Do you want to enter another number? Y/N
     ")
#after the loop ends it outputs the total
print("The total of your numbers was" + str(total))
```

Discuss your output with your lab tutor, SLA or classmate.

Quick Background

Python has a modular structure, it means that it offers basic functions out of the box and you can extend its functionality by importing libraries (modules). Thanks to this feature, your programs will execute faster as the program will only have the features you need. Furthermore, this allows the reuse of code written by others (what is the point of reinventing the wheel?).

Python offers two ways of importing modules – you can import the whole module which will include all the functions within it or you can import certain functions from the module. It is really important to think about efficiency and size of your program. If you are going to use only one function from the module, why would you want to import the massive library? To import the whole module, you have to use the “import” command. If you want to import just a couple of functions from a module, you have to use “from” and “import” commands. (see Fig. 3 how to and Fig. 4 how to import specific functions from the module).

Import the whole module

```
import fibo
```

Import specific functions from the module

```
from fibo import fib, fib2
```

To use a function from an external module you have to use module_name.function_name.

```
# module_name = fibo, function_name = fib, parameter = 100
fibo.fib(100)
```

You don't have to specify module_name each time you run it. If you are using a certain function frequently it might be worth assigning it to a variable. Next time you will want to invoke the module, you can just use the variable name with optional parameters.

```
# Assign external function to a variable
myFib = fibo.fib
# Call the function with parameter specified in brackets
myFib(100)
```

Collections in Python

Python offers 4 types of collections – Lists, Tuples, Sets and Dictionaries. Each collection has some specific properties (see below).

• List – an ordered collection of elements, changeable, allows duplicates.
• Tuple – an ordered collection of elements, unchangeable, allows duplicates.
• Set – an unordered, unindexed collection of elements, does not allow duplicates.
• Dictionary – an unordered, changeable, indexed collection of elements, does not allow duplicates.

Tuple elements can be indexed exactly as list elements, but tuples are immutable: they cannot be changed once defined. They are convenient when you have a fixed-length sequence of elements that will never change (e.g. an offset for a coordinate transformation).

Each of the collections have specific functions related to them. Operations on Lists will be different to operations on Tuples – below you can find operations related to lists and tuples.

Lists – Basic Operations

Python Expression	Results	Description
len([1, 2, 3])	3	Length
[1, 2, 3] + [4, 5, 6]	[1, 2, 3, 4, 5, 6]	Concatenation
[3] * 4	[3, 3, 3, 3]	Repetition
3 in [1, 2, 3]	True	Membership
min([1,2,3])	1	Minimum
max([1,2,3])	3	Maximum

Lists – Indexing and Slicing

L = [1, 2, 3]

Python Expression	Results	Description
L[2]	3	List starts at zero
L[-2]	2	Negative: count from the right (last element = -1)
L[1] = 4	[1, 4, 3]	Change an element
L[1:]	[2, 3]	Slicing remainder of list, including indexed element
L[:-1]	[1, 2]	Slicing start of list, excluding indexed element
L[1:-1]	[2]	Slicing section of list

Lists – Manipulation

L = [1, 2, 3] indexes in blue

Python Expression	Results	Description
L.append(4)	[1, 2, 3, 4]	Append element
L.count(3)	1	Count element
L.extend([4, 5])	[1, 2, 3, 4, 5]	Append each element of the new list append([4,5])
L.index(3)	2	Index of 1st occurrence
L.insert(1,4)	[1, 4, 2, 3]	Insert element at index
L.remove(2)	[1, 3]	Remove element (1st)
L.pop(2)	[1, 2]	Delete and return element by index

Tuples – Operations

```
# creating a tuple
leds = (“ledPrinter”, “ledComputer”, “ledSpeakers”)
# printing the second element of the tuple
print(leds[1])
# it’s not going to work -> tuples are unchangeable
leds[0] = “ledPlotter”
# prints the length of the tuple
print(len(leds))
# deletes the tuple
del leds
# another tuple
leds = tuple(([“ledPrinter”, “ledComputer”, “ledSpeakers”))
```

Converting List to Tuple

```
# lets initiate our list called “leds”
leds = [“ledPrinter”, “ledComputer”, “ledSpeakers”]
# prints list “leds”
print(leds)
# stores “leds” list in a tuple called “ledsTuple” which is unchangeable
ledsTuple = tuple(leds)
# prints tuple “ledsTuple”
print(ledsTuple)
# lets initiate our tuple called “ledsTuple”
ledsTuple = (“ledPrinter”, “ledComputer”, “ledSpeakers”)
# prints tuple “ledsTuple”
print(ledsTuple)
# stores “ledsTuple” tuple in a list called “ledsList” which is changeable
ledsList = list(ledsTuple)
# prints list “ledsList”
print(ledsList)
```

Conditional statements

Python uses conditional statements known as if…else statements. A conditional statement validates if a set condition is true. If it is, it executes the code within the statement. If the condition is not met it either skips the code within the statement or it moves to else (if an else exists). The else statement is triggered only when the condition within the if statement returns false.

Python supports various logical operands as the following shows:

```
Equals: x == y
Not equals: x != y
Less than: x < y
Lest than or equal to: x <= y
Greater than: x > y
Greater than or equal to: x >= y
```

Conditional statements in Python are represented with the “if” command

```
If ledStatus == 1:
     print(ledName)
```

And if…else conditions

```
if ledStatus == 1:
     print(ledName)
else
     print(“Sorry mate, not today”)
```

If statements support logical operators to join multiple conditions together. You can “use”, “and” and “not” to trigger actions depending on multiple conditions linked together.

Logical operators

Operator	Description	Example
and Logical AND	If both the operands are true then condition becomes true.	(a and b) is true.
or Logical OR	If any of the two operands are non-zero then condition becomes true.	(a or b) is true.
not Logical NOT	Used to reverse the logical state of its operand.	Not(a and b) is false.

Loops

Python supports multiple types of looping mechanisms. You are going to use two of them called “while” and “for”. Loops are used to perform repeatable tasks based on the condition.

The while loop repeats a statement or group of statements while the set condition is TRUE. The following presents how to use a while loop, the code should print the value of ledCount 4 times.

```
ledCount = 1
while ledCount < 5:
     print(ledCount)
     # same as ledCount += 1
     ledCount = ledCount + 1
```

For loops iterate through sequences e.g. lists, tuples, sets, dictionaries or Strings. Opposingly to while loops, for loops do not validate conditions. The following presents an example usage of a for loop, the code should print each element of the “leds” list.

```
leds = [“ledPrinter”, “ledComputer”, “ledSpeakers”]
for x in leds:
     print(x)
```

For loops can iterate through Strings – it will read one character of a String in each iteration of the loop. For loops can iterate through the output of the range() function – a function that generates a list of numbers in a certain range for set parameters. By default, it increments by 1 at the end of each iteration, however it can be changed with

an additional parameter. Below you can find some examples of the range() function used in for loops.

For loop iterating through range(10) - it will print numbers from 0 to 9

```
for x in range(10):
     print(x)
```

For loop iterating through range(5,10) - it will print numbers from 5 to 9

```
for x in range(5,10):
     print(x)
```

For loop iterating through range(2,10,2) - it will print numbers from 2 to 8 incrementing by 2 at each iteration

```
for x in range(2,10,2):
     print(x)
```

For loop iterating through range(5,10) function - it will print numbers from 5 to 9 and content of else statement after it completes execution

```
for x in range(5, 10):
     print(x)
else:
     print("I completed the execution")
```

Nested For Loops

For loops can be nested within others. The inner loop will complete all of its iterations every time the outer loop iterates once.

Example of nested for loop

```
for x in range(2):
     for y in range(3):
          print(x, y)
```

Expected Output

0 0
0 1
0 2
1 0
1 1
1 2

In other words, nesting a loop means simply having a loop (let's call it the 'outer loop') that has inside its commands another loop (let's call it the 'inner loop'). For example, the code appearing below shows two nested loops, an outer for loop over the values of i and an inner for loop over the values of j to multiply inside the inner loop all nine elements of a 3x3 matrix A by a factor f that changes according to the outer loop iteration.

```
A = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
f = 1
print(A)
for i in range(0, 3)
     f *= 10
     for j in range(0, 3):
          A[i][j] *= f
print(A)
```

The expected output:

[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
[[10, 20, 30], [400, 500, 600], [7000, 8000, 9000]]

Therefore, the outer loop executes 3 iterations (i equal to 0, 1, and 2), and at each iteration the following is executed:

• The multiplication of the factor f by 10
• The execution of the inner loop that has 3 iterations (j equal to 0, 1, and 2), and at each iteration the element i,j of A is multiplied by f

EX 7: Creating Python code to execute arithmetic operations

Write a program that will use two variables, x and y, that are hardcoded. The program should check if x is greater than y. If it is, then it should perform the following addition x+y and print the result to the user. If x is smaller than or equal to y, then it should perform the following subtraction y-x and print the result to the user.

EX 8: Creating Python code to execute nested for loops

Write a program that will utilise nested for loops. The outer loop will iterate through a list of integers called num_list, and the inner loop will iterate through a list of strings called alpha_list. Your inputs should be:

```
num_list = [1, 2, 3]
alpha_list = ['a', 'b', 'c']
```

The expected output should look like the following:

```
1
a
b
c
2
a
b
c
3
a
b
c
```

The output illustrates that the program completes the first iteration of the outer loop by printing 1, which then triggers completion of the inner loop, printing a, b, c consecutively. Once the inner loop has completed, the program returns to the top of the outer loop, prints 2, then again prints the inner loop in its entirety (a, b, c), etc.

EX 9: Creating basic code in Python to execute conditional statements with looping

Write a program that will take an integer number input from the user. The program has to check if the number is greater than 0, if it is it should ask for another integer number. If the second number is greater than 1 it should take the first number to the power of the second number – this task must be accomplished using loops.

EX 10: Performing advanced code in Python to store results in a list

Modify the program from the previous exercise to store the results in a list called results. Print the list to the user.

Continue for EX11 >>>

EX 11: Performing complex task in Python

You are given the following piece of Python code – investigate what the issue is and fix the errors.

```
#!/usr/bin/Python

i = 0
nameList = ["Eris", "Denis", "Igor"]

###### FOR + IF ######
print("Printing even indexes with for loop")
for x in nameList:
     if i % 2 == 0:
          print("Current   i   value\t",   i,   "\tCurrent   loop
          iteration\t", x)
     i += 1

i = 0
pirnt("\n\nPrinting odd indexes with for loop")
for x in nameList:
     if i % 2 == 1:
print("Current i value\t", i, "\tCurrent loop iteration\t", x)
     i += 1

###### WHILE + IF ######
print("\n\nPrinting even indexes with while loop")
i = 0
while i <= len(nameList):
     if i % 2 == 0:
          print("Current   i   value\t",   i,   "\tCurrent   loop
          iteration\t", x
     i += 1

print("\n\nPrinting odd indexes with while loop")
i = 0
while i < len(nameList):
     if i % 2 == 1:
          print("Current   i   value\t",   i,   "\tCurrent   loop
          iteration\t", x)
     i += 1
```